

PPL Lecture-3

Data Storage/Representation,
Flowcharts and Algorithms

Data Storage/Representation

- **Data** refers to the symbols that represent people, events, things, and ideas.
- Data can be a name, a number, the colors in a photograph, or the notes in a musical composition.
- **Data Representation** refers to the form in which data is stored, processed, and transmitted.
- Devices such as smartphones, iPods, and computers store data in digital formats that can be handled by electronic circuitry
- **Digitization** is the process of converting information, such as text, numbers, photo, or music, into digital data that can be manipulated by electronic devices.

Data Representation

0

ASCII codes
represent data as
0s and 1s

1

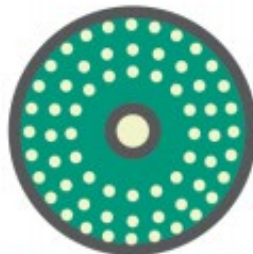
1 1 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0



Circuit boards
carry data as
pulses of current



+5 +5 -2 +5 -2 -2 -2 +5 -2 +5 -2 +5 -2 +5 -2 -2 -2



CDs and DVDs
store data as dark
and light spots



• • • • • • • • • • • • • • • • • • • •



Disk drives store
data as magnetized
particles



+ + - + - - - + - + - + - - -

Representing Numbers

- Numeric data consists of numbers that can be used in arithmetic operations.
- Digital devices represent numeric data using the binary number system, also called base 2.
- The binary number system only has two digits: 0 and 1.
 - No numeral like 2 exists in the system, so the number “two” is represented in binary as 10 (pronounced “one zero”).
- Other types:
 - Decimal Number System: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Octal Number System: 0, 1, 2, 3, 4, 5, 6, 7
 - Hexadecimal Number System: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Representing Numbers

| DECIMAL (BASE 10) | BINARY (BASE 2) |
|-------------------|-----------------|
| 0 | 0 |
| 1 | 1 |
| 2 | 10 |
| 3 | 11 |
| 4 | 100 |
| 5 | 101 |
| 6 | 110 |
| 7 | 111 |
| 8 | 1000 |
| 9 | 1001 |
| 10 | 1010 |

https://www.youtube.com/watch?v=pLTDDsvMnFQ&ab_channel=IITKharagpurJuly2018

Representing Text

- **Character data** is composed of letters, symbols, and numerals that are not used in calculations.
- Examples of character data include your name, address, and hair color.
- Character data is commonly referred to as “**text**”.
- Digital devices employ several types of codes to represent character data, including ASCII, Unicode, and their variants.
- **ASCII** (American Standard Code for Information Interchange, pronounced “ASK ee”) requires seven bits for each character.
- The ASCII code for an uppercase A is 1000001.

ASCII Table

| Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char | Decimal | Hex | Char |
|---------|-----|------------------------|---------|-----|---------|---------|-----|------|---------|-----|-------|
| 0 | 0 | [NULL] | 32 | 20 | [SPACE] | 64 | 40 | @ | 96 | 60 | ` |
| 1 | 1 | [START OF HEADING] | 33 | 21 | ! | 65 | 41 | A | 97 | 61 | a |
| 2 | 2 | [START OF TEXT] | 34 | 22 | " | 66 | 42 | B | 98 | 62 | b |
| 3 | 3 | [END OF TEXT] | 35 | 23 | # | 67 | 43 | C | 99 | 63 | c |
| 4 | 4 | [END OF TRANSMISSION] | 36 | 24 | \$ | 68 | 44 | D | 100 | 64 | d |
| 5 | 5 | [ENQUIRY] | 37 | 25 | % | 69 | 45 | E | 101 | 65 | e |
| 6 | 6 | [ACKNOWLEDGE] | 38 | 26 | & | 70 | 46 | F | 102 | 66 | f |
| 7 | 7 | [BELL] | 39 | 27 | ' | 71 | 47 | G | 103 | 67 | g |
| 8 | 8 | [BACKSPACE] | 40 | 28 | (| 72 | 48 | H | 104 | 68 | h |
| 9 | 9 | [HORIZONTAL TAB] | 41 | 29 |) | 73 | 49 | I | 105 | 69 | i |
| 10 | A | [LINE FEED] | 42 | 2A | * | 74 | 4A | J | 106 | 6A | j |
| 11 | B | [VERTICAL TAB] | 43 | 2B | + | 75 | 4B | K | 107 | 6B | k |
| 12 | C | [FORM FEED] | 44 | 2C | , | 76 | 4C | L | 108 | 6C | l |
| 13 | D | [CARRIAGE RETURN] | 45 | 2D | - | 77 | 4D | M | 109 | 6D | m |
| 14 | E | [SHIFT OUT] | 46 | 2E | . | 78 | 4E | N | 110 | 6E | n |
| 15 | F | [SHIFT IN] | 47 | 2F | / | 79 | 4F | O | 111 | 6F | o |
| 16 | 10 | [DATA LINK ESCAPE] | 48 | 30 | 0 | 80 | 50 | P | 112 | 70 | p |
| 17 | 11 | [DEVICE CONTROL 1] | 49 | 31 | 1 | 81 | 51 | Q | 113 | 71 | q |
| 18 | 12 | [DEVICE CONTROL 2] | 50 | 32 | 2 | 82 | 52 | R | 114 | 72 | r |
| 19 | 13 | [DEVICE CONTROL 3] | 51 | 33 | 3 | 83 | 53 | S | 115 | 73 | s |
| 20 | 14 | [DEVICE CONTROL 4] | 52 | 34 | 4 | 84 | 54 | T | 116 | 74 | t |
| 21 | 15 | [NEGATIVE ACKNOWLEDGE] | 53 | 35 | 5 | 85 | 55 | U | 117 | 75 | u |
| 22 | 16 | [SYNCHRONOUS IDLE] | 54 | 36 | 6 | 86 | 56 | V | 118 | 76 | v |
| 23 | 17 | [ENG OF TRANS. BLOCK] | 55 | 37 | 7 | 87 | 57 | W | 119 | 77 | w |
| 24 | 18 | [CANCEL] | 56 | 38 | 8 | 88 | 58 | X | 120 | 78 | x |
| 25 | 19 | [END OF MEDIUM] | 57 | 39 | 9 | 89 | 59 | Y | 121 | 79 | y |
| 26 | 1A | [SUBSTITUTE] | 58 | 3A | : | 90 | 5A | Z | 122 | 7A | z |
| 27 | 1B | [ESCAPE] | 59 | 3B | ; | 91 | 5B | [| 123 | 7B | { |
| 28 | 1C | [FILE SEPARATOR] | 60 | 3C | < | 92 | 5C | \ | 124 | 7C | |
| 29 | 1D | [GROUP SEPARATOR] | 61 | 3D | = | 93 | 5D |] | 125 | 7D | } |
| 30 | 1E | [RECORD SEPARATOR] | 62 | 3E | > | 94 | 5E | ^ | 126 | 7E | ~ |
| 31 | 1F | [UNIT SEPARATOR] | 63 | 3F | ? | 95 | 5F | _ | 127 | 7F | [DEL] |

Programming Environment

- A programming environment consists of:
 - A text editor to create computer programs.
 - A compiler to compile the programs into binary format.
 - An interpreter to execute the programs directly.
- **Text Editor**
 - A text editor is a software that is used to write computer programs.
 - Examples: Notepad, WordPad, gedit, vi editor
- An **IDE (Integrated Development Environment)** is a single application that integrates all the tools to perform those steps.
 - Examples: CodeBlocks, NetBeans, Eclipse, Atom

Debugging

- To ***debug*** a program or hardware device is to start with a problem, isolate the source of the problem, and then fix it.
- Process of identifying, isolating and correcting the errors
- **Two Categories:**
 - **Syntax errors** -- Errors in spelling and grammar.
 - We can use the compiler or interpreter to uncover syntax errors.
 - We must have a good working knowledge of error messages to discover the cause of the error.
 - Example: C line -> printf("Hello!")
 - **Logical errors** -- Errors that indicate the logic used when coding the program failed to solve the problem.
 - We do not get error messages with logic errors.
 - Our only clue to the existence of logic errors is the production of wrong solutions.

Algorithms and Flowcharts

- A typical programming task can be divided into two phases:
 1. *Problem solving phase*
 - produce an ordered sequence of steps that describe solution of problem
 - this sequence of steps is called an ***algorithm***
 2. *Implementation phase*
 - implement the program in some programming language

Steps in Problem Solving

- First produce a general algorithm (one can use *pseudocode*)
- Refine the algorithm successively to get step by step detailed *algorithm* that is very close to a computer language.
- *Pseudocode* is an artificial and informal language that helps programmers develop algorithms.
- Pseudocode is very similar to everyday English.

Pseudocode & Algorithm

- **Example:** Write an algorithm to determine a student's final grade and indicate whether it is passing or failing. The final grade is calculated as the average of four marks.

Pseudocode & Algorithm

Pseudocode:

- *Input a set of 4 marks*
- *Calculate their average by summing and dividing by 4*
- *If average is below 30*
 Print "FAIL"
Else
 Print "PASS"

Pseudocode & Algorithm

Detailed Algorithm

Step 1: Input M1, M2, M3, M4

Step 2: $\text{GRADE} \leftarrow (M1+M2+M3+M4)/4$

Step 3: if (GRADE < 30) then

 Print "FAIL"

 else

 Print "PASS"

 endif

The Flowchart

- **Dictionary**

- A schematic representation of a sequence of operations, as in a manufacturing process or computer program.

- **Technical**

- A graphical representation of the sequence of operations in an information system or program.
- Information system flowcharts show how data flows from source documents through the computer to final distribution to users.
- Program flowcharts show the sequence of instructions in a single program or subroutine.
- Different symbols are used to draw each type of flowchart.

The Flowchart

A Flowchart

- shows logic of an algorithm
- emphasizes individual steps and their interconnections
- e.g. control flow from one action to the next

Flowchart Symbols

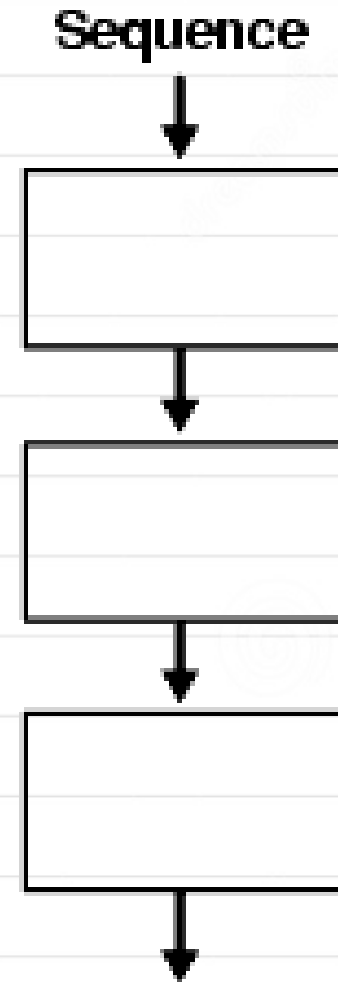
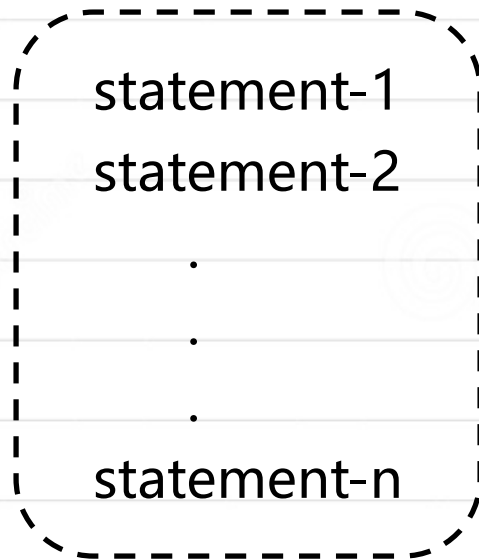
| Symbol | Name | Function |
|---|--------------|--|
|  | Start/End | Flowchart terminator |
|  | Arrows | A line is a connector that represent relationship between the representative shapes and its flow direction |
|  | Input/Output | A parallelogram represents input or output of your program |
|  | Process | A rectangle represent a process |
|  | Decision | A diamond indicates a decision |

Control Structures in Programming Languages

- **Control Structures** are a way to specify flow of control in programs
- There are three basic types of logic, or flow of control:
 1. Sequence logic, or sequential flow
 2. Selection logic, or conditional flow
 3. Iteration logic, or repetitive flow

Sequential Logic (Sequential Flow)

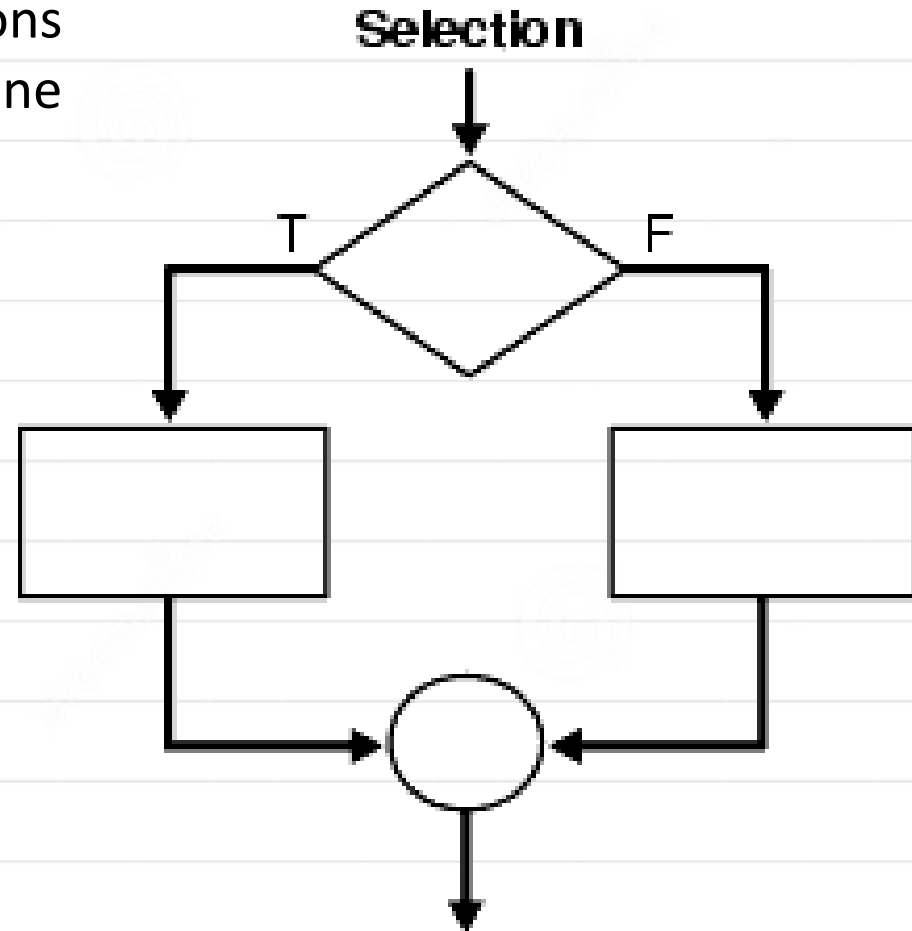
- It follows a serial or sequential flow in which the flow depends on the series of instructions given to the computer.



Selection Logic (Conditional Flow)

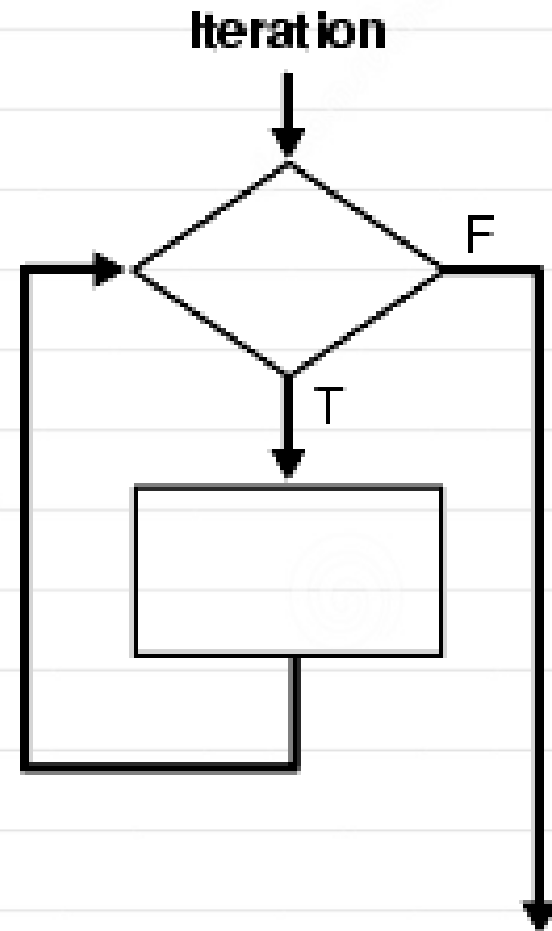
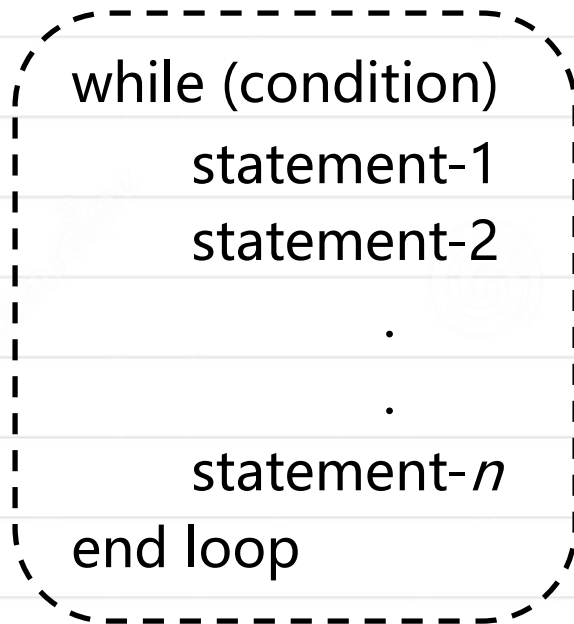
- It involves a number of conditions or parameters which decides one out of several written modules.

```
if (condition)  
    statement-1  
else  
    statement-2  
endif
```



Iteration Logic (Repetitive Flow)

- It employs a loop which involves a repeat statement followed by a module known as the body of a loop.



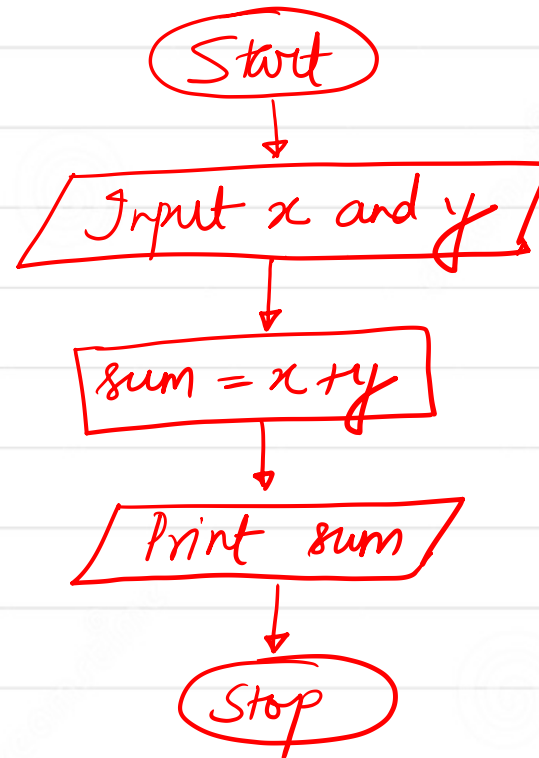
Example 1

Add two numbers

Step1. Input x and y

Step2. $\text{sum} = x + y$

Step3. Print sum



Example 2

Swap two numbers

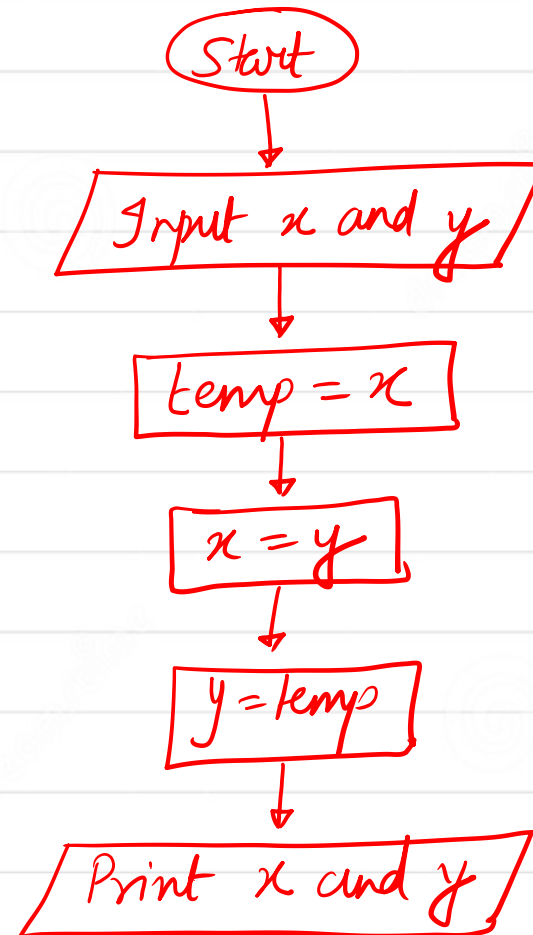
Step1. Input x and y

Step2. temp = x

Step3. x = y

Step4. y = temp

Step5. Print x and y



Example 3

Check whether a given number is even or odd

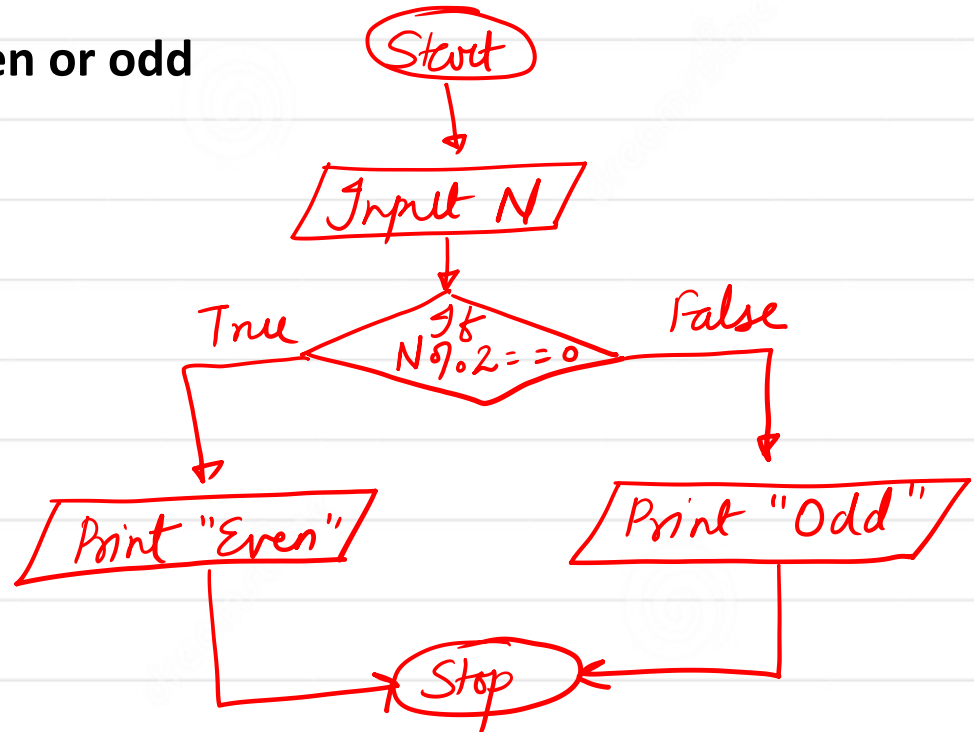
Step 1: Input N

Step 2: if ($N \% 2 == 0$) then
Print "Even"

else

Print "Odd"

endif



Example 4

Find grade based on marks of four subjects

Step 1: Input M1,M2,M3,M4

Step 2: $GRADE \leftarrow (M1+M2+M3+M4)/4$

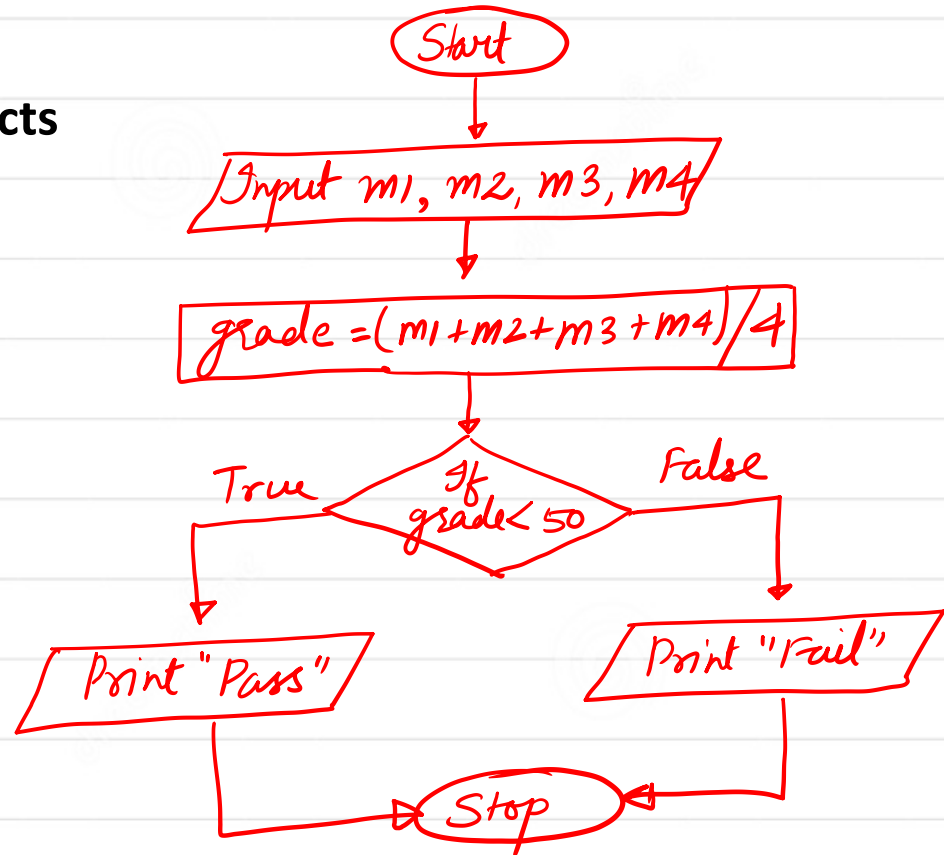
Step 3: if (GRADE < 50) then

Print "FAIL"

else

Print "PASS"

endif



Example 5

Find roots of a quadratic equation

Step 1: Input coefficients a, b, c

Step 2: $d = (b*b) - (4*a*c)$

Step 3: if ($d < 0$) then

Print "Roots are imaginary"

else if ($d == 0$)

$x1 = -b / (2*a)$

$x2 = -b / (2*a)$

Print x1 and x2

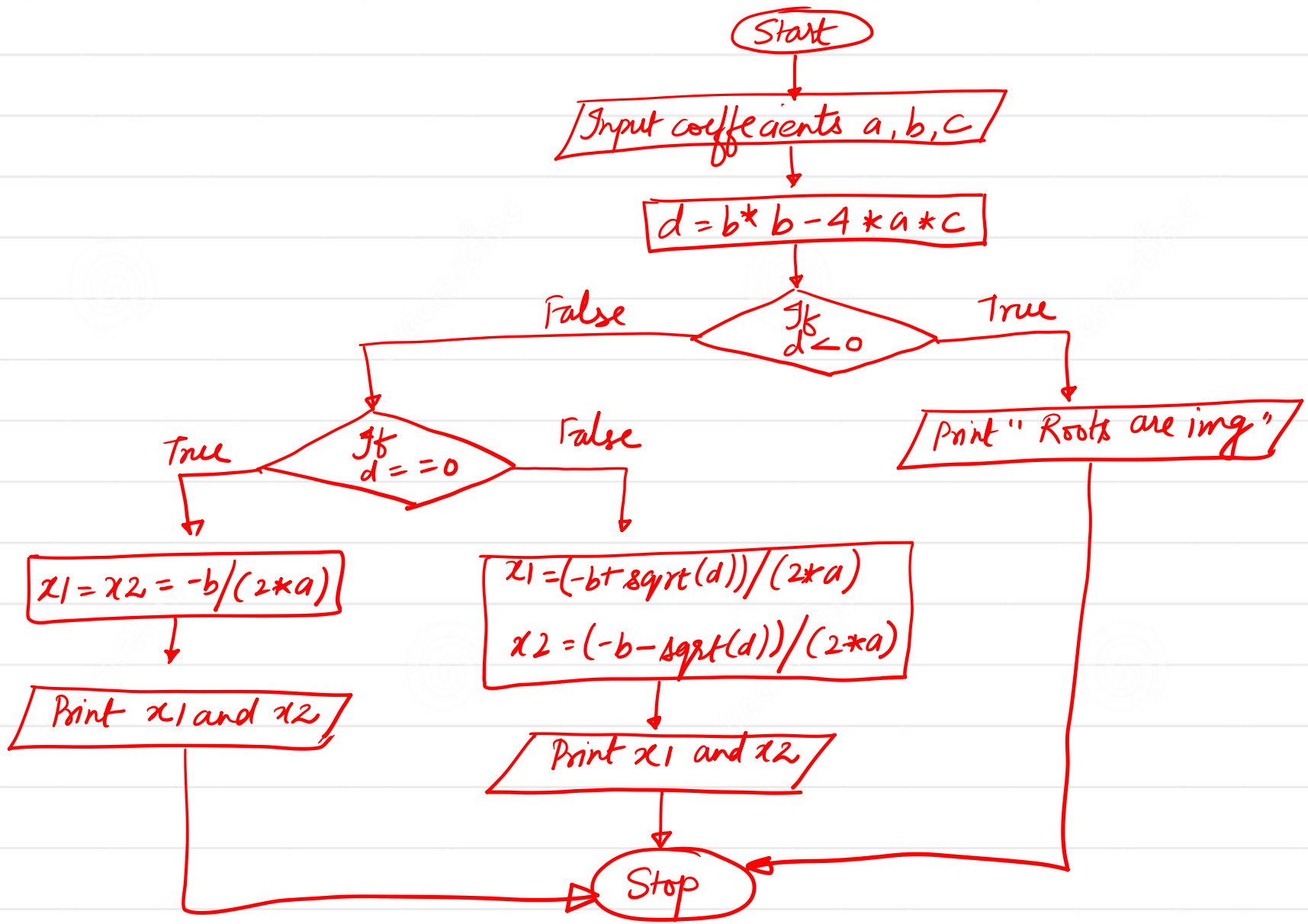
else

$x1 = (-b + \sqrt{d}) / (2*a)$

$x1 = (-b - \sqrt{d}) / (2*a)$

Print x1 and x2

endif



Example 6

Calculate sum of first n natural numbers.

Step 1: Input n

Step 2: $\text{sum} = 0, i = 1$

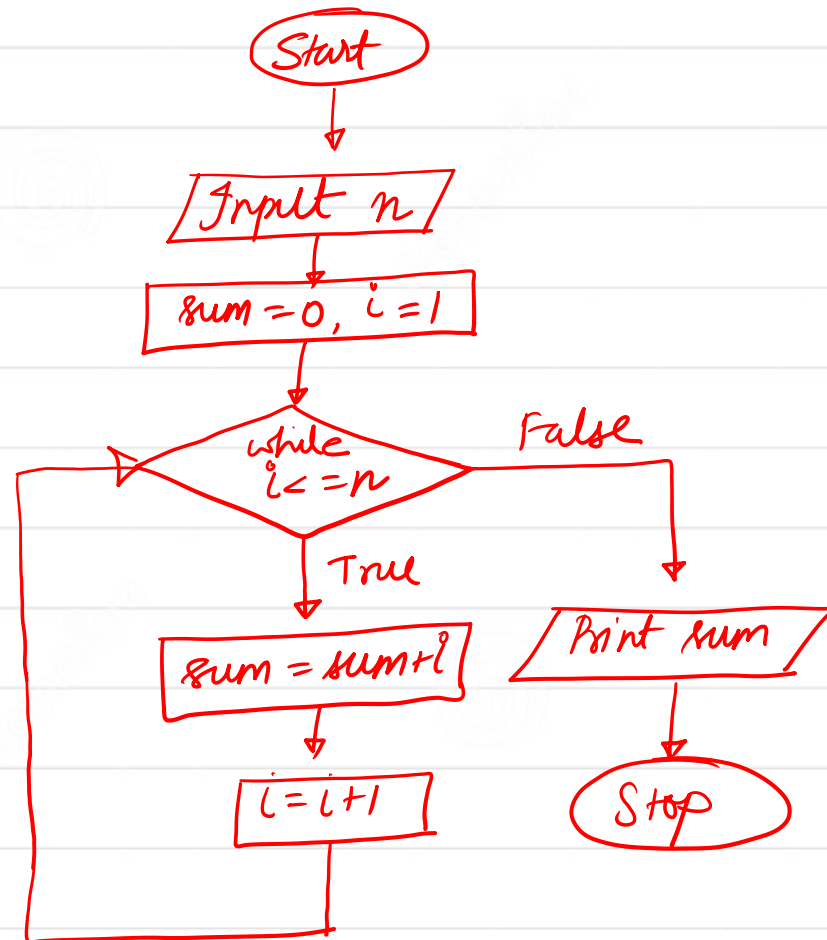
Step 3: while ($i \leq n$)

$\text{sum} = \text{sum} + i$

$i = i + 1$

end while

Step 4: Print sum



Example 7

Print first n terms of Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21,

Step 1: Input n

Step 2: $\text{first_term} = 0, \text{sec_term} = 1$

Step 3: Print the value of first_term and sec_term

Step 4: $i = 1$

Step 5: while ($i < n - 2$)

$\text{third_term} = \text{first_term} + \text{sec_term}$

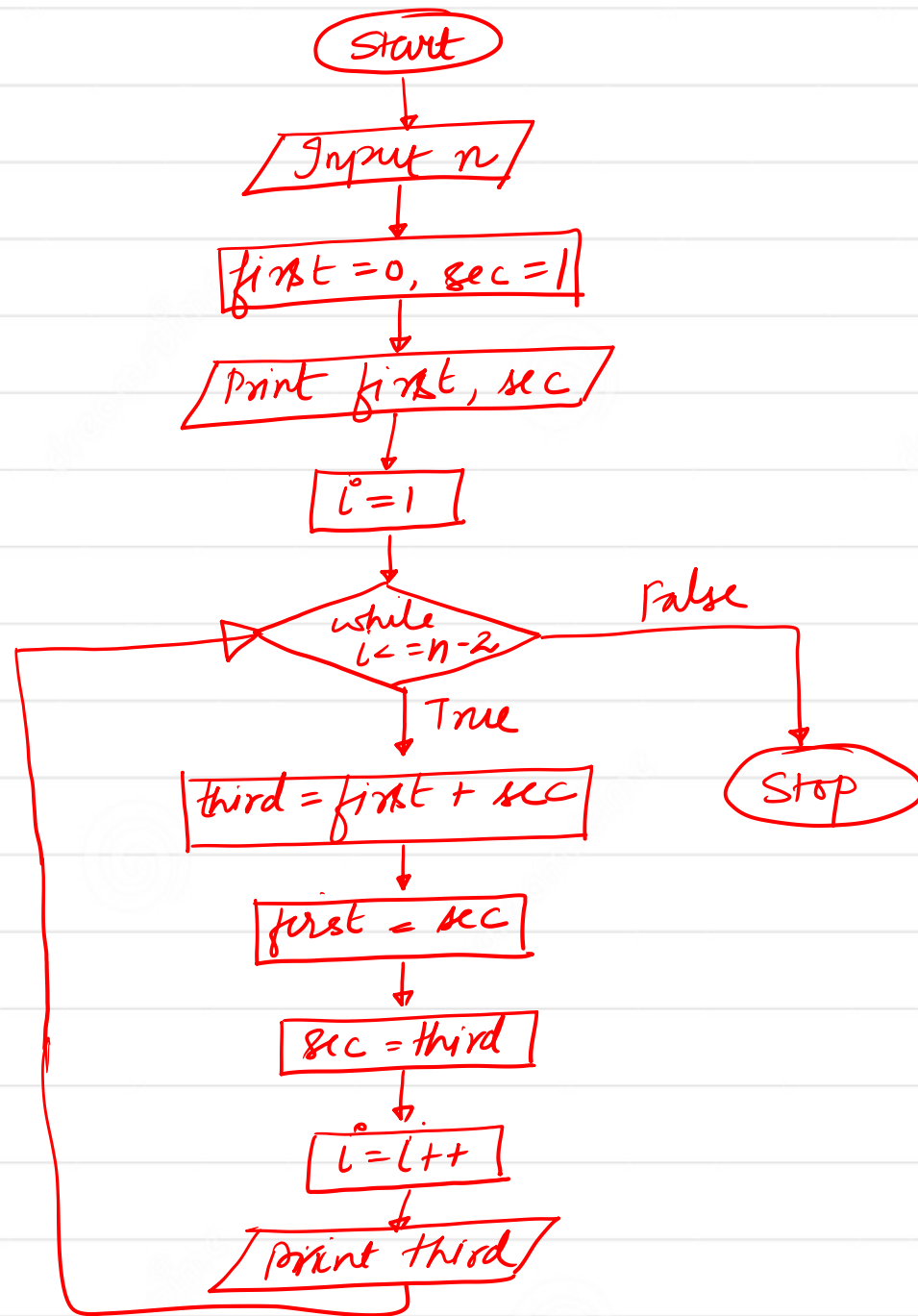
$\text{first_term} = \text{sec_term}$

$\text{sec_term} = \text{third_term}$

 Print third_term

$i = i + 1$

end while



Example 8

Calculate the sum of digits of a number.

Ex: $N = 3275 \Rightarrow \text{Sum of digits} = 3+2+7+5 = 17$

Step 1: Input N

Step 2: $\text{sum} = 0$

Step 3: while ($N > 0$)

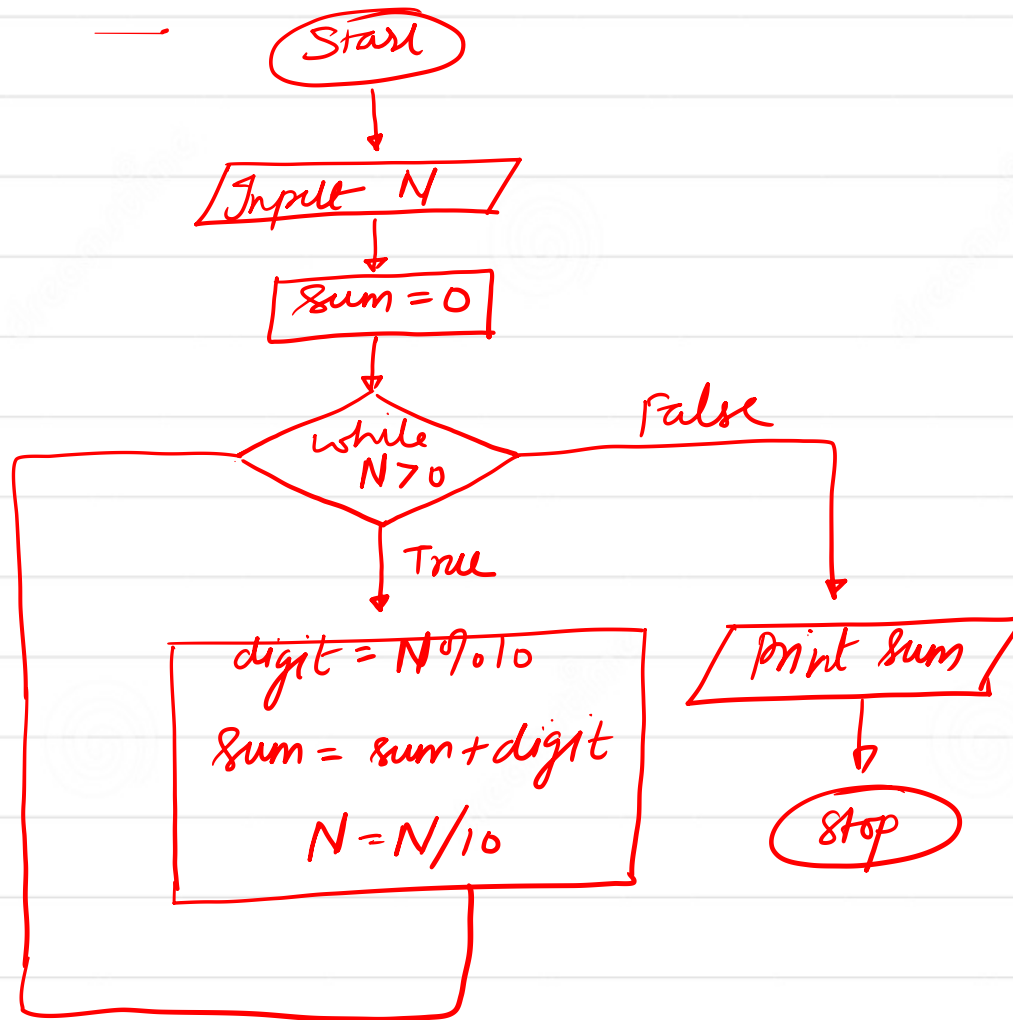
$\text{digit} = N \% 10$

$\text{sum} = \text{sum} + \text{digit}$

$N = N / 10$

end while

Step 4: Print sum



Example 9

Check whether a number is prime or not

Step 1: Input N

Step 2: $i = 2$, $\text{flag} = 0$

Step 3: while ($i < N/2$)

 if ($N \% i == 0$)

$\text{flag} = 1$

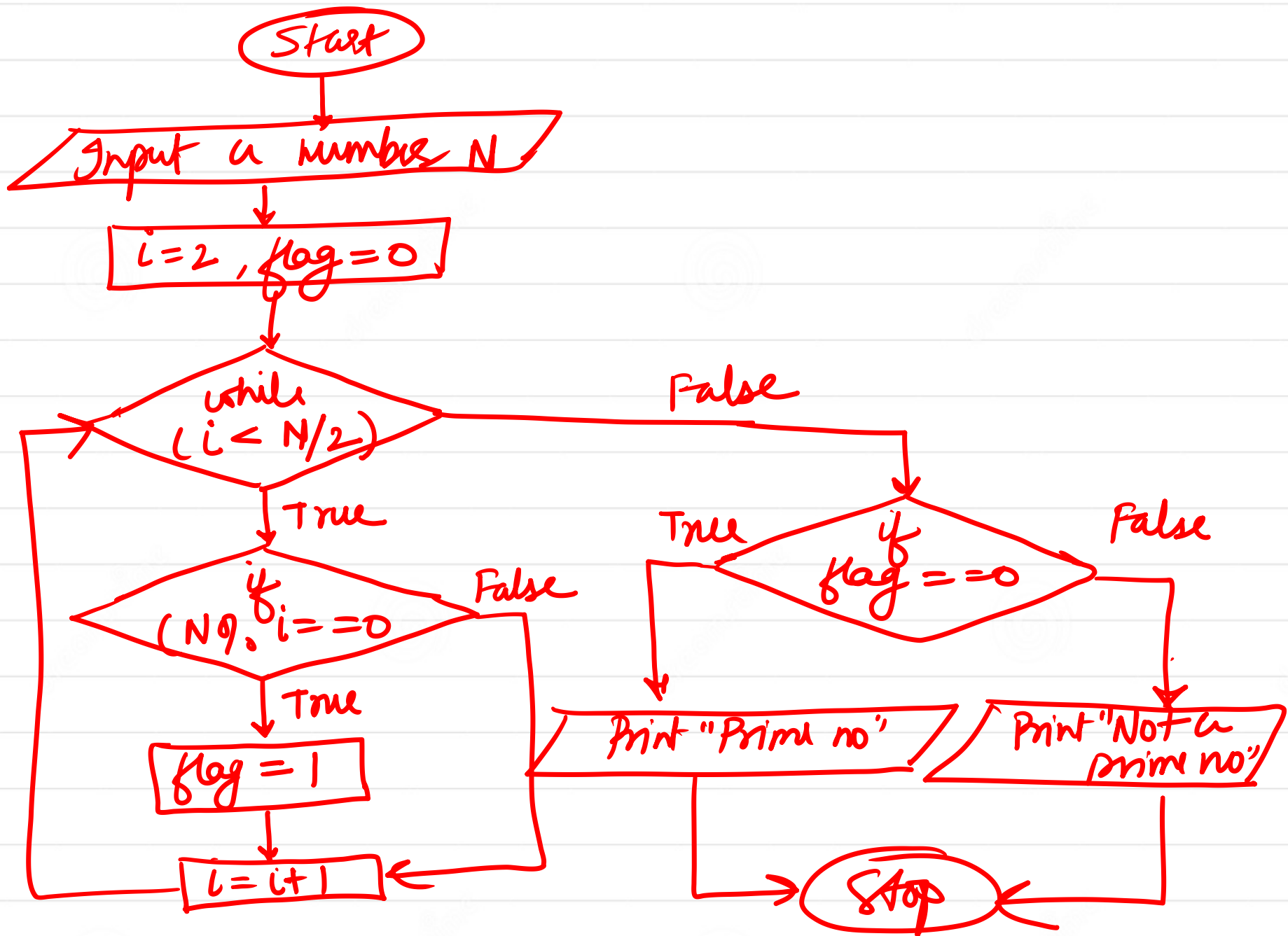
$i = i + 1$

Step 4: if ($\text{flag} == 0$)

 Print "Prime number"

else

 Print "Not a prime number"



Example 10

Find GCD of two numbers

Step 1: Input two numbers $n1, n2$

Step 2: while ($n2 \neq 0$)

$temp = n2$

$n2 = n1 \% n2$

$n1 = temp$

end while

Step 3: Print "GCD =" $n1$

